P2782R0
A proposal for a type trait to detect if value initialization can be achieved by zero-filling

Published Proposal, 2023-01-30

Author:
Giuseppe D'Angelo

Audience:
EWG, LEWG, SG14

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose a new type trait that checks whether an instance of a trivially default constructible type can be value-initialized by setting all the bytes in its object representation to 0. The trait helps with detecting implementation-specific behavior: the object representation of floating-point types, pointer types, pointer to non-static data members etc. is not mandated by the Standard and ultimately depends on the platform. We therefore need compiler hooks in order to implement this detection. The detection offered by this trait can be used to optimize the implementation of certain functions and algorithms.

Table of Contents

§ 1. Changelog

- R0
  - First submission

§ 2. Motivation and Scope

Consider the `std::uninitialized_value_construct` algorithm. This algorithm constructs a number of objects into uninitialized storage, value-initializing each object. A crude implementation looks like this:

```
template<class ForwardIt>
void uninitialized_value_construct(ForwardIt first, ForwardIt last)
{
    using Value = typename std::iterator_traits<ForwardIt>::value_type;
    ForwardIt current = first;

    try {
        for (; current != last; ++current)
            ::new std::addressof(*current) Value();
    } catch (...) {
        std::destroy(first, current);
        throw;
    }
}
```

This algorithm has a natural application in containers, since they have operations that require to value-initialize elements in bulk (such as `resize(N)`, or a constructor such as `container<X> obj(N)`).

(Technically speaking, allocator-aware containers cannot use the algorithm directly, because they are supposed to use `std::allocator_traits<Alloc>::construct`; we'll come back to this in a second.)

There’s an enormous performance improvement possible in case we need to value-initialize objects of a "simple" datatype, for instance `int`, and we’re constructing over contiguous storage (f.i. `vector<int>`). In this case, the `for` loop can be entirely replaced by a completely equivalent call to `memset(ptr, 0, bytes_size);`. In case we’re acquiring brand new storage, it could be acquired using `calloc` instead of `malloc`.

It turns out that optimizing *compilers* already do this transformation. For instance, GCC 12 has this codegen on X86-64:

```
#include <new>

using T = int;
extern T buffer[];
extern std::size_t N;

int main() {
    for (std::size_t i = 0; i < N; ++i)
        ::new (buffer + i) T();
}
```

```
main:
        mov     rdx, QWORD PTR N[rip]
        test    rdx, rdx
        je      .L7
        sub     rsp, 8
        sal     rdx, 2
        xor     esi, esi
        mov     edi, OFFSET FLAT:buffer
        call    memset
        xor     eax, eax
        add     rsp, 8
        ret
.L7:
        xor     eax, eax
        ret
```

Compiler Explorer shows that GCC 12, Clang 15, MSVC "latest" all implement this optimization.

The branch in the generated code exists to avoid potentially passing `nullptr` to `memset`, which is undefined behavior (even if `N` is 0). Adding a compiler assumption on `N > 0` makes the branch disappear.

This optimization is extremely advantageous; amongst other things, as mentioned before, an allocator-aware container needs to add a further indirection to construct each element. An optimizer can "see through" all the relevant code and replace a `construct` loop with much more efficient code.

However, relying on the optimizer comes with the usual set of problems:

- compilers sometimes miss the transformation and leave the loop in the code (example of GCC missing the optimization);
- one needs aggressive optimizations turned on (under GCC, at least -O2 is necessary);
- optimizations hurt the debugging experience, and disabling optimizations leads to very inefficient code generation, which *also* hurts debugging (lose/lose scenario);
- optimizations increase compilation times, and this hurts the development cycle.

For these reasons, many libraries manually implement the optimization above in their source code. In other words, if they can detect that it’s "safe" to zero-fill memory in order to perform value initialization for a given type `T`, then they will explictly call `memset`.

What this paper proposes is a type trait that implements this detection so that it is correct and complete. Such a trait is currently lacking from the Standard Library.

§ 2.1. Prior art

Usage of `memset` to zero-fill in order to achieve value initialization happens for instance in Boost.Container (in spite of the containers being allocator-aware!); in FBVector from Folly through the `IsZeroInitializable` type trait; and used to happen in Qt container classes (which are not allocator-aware).

Since there isn't a standard type trait that detects if zero-filling is possible for a type `T`, all of these libraries use an ad-hoc detection, which is incomplete and, in many cases, incorrect. Specifically:

- Boost.Container uses zero-filling by default on integer types, floating point types, pointers to object and functions (à la `is_pointer` -- excluding pointers to data members / member functions), and before Boost 1.82, also POD types. There are opt-outs available using certain preprocessor macros.
- Folly uses zero-filling on non-class types by default. There is an opt-in available (specializing the `IsZeroInitializable` trait).
- Qt's contiguous containers (e.g. QVector) used to zero-fill trivial types, and an opt-in was offered through a type trait (`Q_PRIMITIVE_TYPE`). Today, the containers do not longer zero-fill (for the reasons discussed below); certain type erasure facilities zero-fill only scalar types that aren't pointers to members.

This detection is clearly *incomplete*:

- a non-POD type (e.g. a non-standard-layout one) such as `class C { public: int x; private: int y; };` is in principle zero-fillable, but Boost.Container classes won't use `memset` on it;
- the same type won't be zero-filled automatically by `FBVector`, unless one enables the corresponding trait (`C` is a class type);
- and finally the same type won't be zero-filled by Qt, which no longer considers any user-defined type as zero-fillable.

Is the detection even correct?

- All three libraries correctly detect integers (a zero-filled representation must give the value 0, which matches value initialization); floating point types (same, assuming IEEE 754 representation); and pointers to objects and pointers to functions (on any common ABI).
- Pointers to data members and member functions are more problematic. For instance, on the Itanium C++ ABI, a pointer to data member *cannot* be zero filled in order to be value initialized; it must instead be initialized with the value -1 (cf. the specification). Boost and Qt correctly handle this case, but **Folly does not**.
- Folly also zero-fills *union* types, even when their value initialization cannot be achieved this way (upstream bug).
- It is impossible to know if a class type can be zero-filled, as a class may contain such a pointer to data member. **Boost erroneously did not exclude class types**, and in fact considered a POD type such as `struct S { int S::*ptr; };` as zero-fillable (upstream bug, fixed in Boost 1.82). On Itanium, as we have just discussed, `S` is

not zero-fillable; the result is that the elements in e.g. a `boost::container::vector<S> v(10);` object have not been correctly value initialized -- their `ptr` members are *not* null pointers (!).

- **Qt has also historically had the very same bug**: the `S` type above is trivial, and Qt used to consider trivial types as zero-fillable. This problem was fixed in general only very recently (December 2022, see here, here, here). In principle, Qt could reintroduce zero-filling in containers using a limited detection.

The conclusion is that creating an ad-hoc detection is incomplete and extremely error prone. *Expert* C++ developers from three major C++ libraries have consistently got it wrong. Moreover, we do not believe that this trait can be fully implemented in user code without some form of compiler support (cf. § 3.1 Do we need this trait in the Standard Library? Can it be implemented entirely in user code?).

These considerations call for adding this trait to the Standard Library.

§ 2.2. Further applications

The trait that we are proposing can also be used as an optimization for type-erased factories.

In order to build a value-initialized instance of a type `T` (identified by some means by the factory -- the name, an id, etc.), the factory would normally need to store a pointer to a "construction function" that performs value initialization for `T` in some storage space. If the factory can detect that `T` can be value-initialized by zero filling, it could store that information somewhere (e.g. alongside `T`'s other metadata such as size, alignment, etc.) and simply use `memset` instead. The construction function for `T` would then *not* be generated at all, and this would reduce code bloat (by generating less code). Qt uses this optimization in QMetaType.

§ 3. Design Decisions

§ 3.1. Do we need this trait in the Standard Library? Can it be implemented entirely in user code?

At the time of this writing we believe that it is *not possible* to implement this trait in a way that is correct and complete without using private compiler hooks. Basically, if a trivially default constructible type `T` contains a pointer to data member, we cannot zero-fill it on Itanium, but there is no way to know if this is the case "from the outside". It is certainly an interesting application of the capabilities of a static reflection system, should C++ gain one.

An interesting idea (many thanks to Ed Catmur) is to try to `bit_cast` a value-initialized instance of type `T` to an array of bytes of suitable size (e.g. `array<unsigned char, sizeof(T)>`). The result can then be checked for bits different from zero, for instance by comparing it against a zero-filled array:

```
template <typename T>
constexpr bool is_value_initialized_to_zero_v = []
{
    using A = std::array<unsigned char, sizeof(T)>;
    return A{} == std::bit_cast<A>(T());
}();
```

This detection can then be combined with checking whether `T` is trivially default constructible.

Note that trivial default constructability implies that `T` has not a user-provided default constructor ([class.default.ctor]/3), which also implies that value initialization performs zero initialization ([dcl.init.general]/9.1.1 and 9.3). If `T` has padding bits, then the provision in [dcl.init.general]/6.2 ensures that they are set to 0 when performing zero initialization. This means that comparing against a zero-filled buffer will work correctly even in the presence of padding bits.

The above snippet however does not work in case `T` contains pointers, as `std::bit_cast` is not `constexpr` in that context ([bit.cast]/3.2 and 3.3). Moreover, and pending [LWG2827] resolution, in general `T` should not be required to be trivially *copyable* (a constraint of `bit_cast`, [bit.cast]/1.3); in fact, `T` should not be required to be trivially destructible at all, but only trivially default constructible.

In principle, the restriction of `bit_cast` on pointers could be relaxed so that constant evaluation works if one asks to cast a *null pointer value*. Assuming we also solve the problem that we don't want to require trivial copiability, we would still be left with a somehow tricky/clever/"experts-only" implementation; wrapping it in a standardized type trait would definitely increase its usability and discoverability.

§ 3.2. What about padding bits?

See the remark in § 3.1 Do we need this trait in the Standard Library? Can it be implemented entirely in user code?.

§ 3.3. Bikeshedding: naming

The trait that we are proposing describes a type property which does not have a pre-existing name in the Standard. We must therefore introduce a new name.

For the moment being, we are going to propose the (quite verbose) "trivially value-initializable by zero-filling" name. This describes all the characteristics that we are looking for:

- we want to achieve value initialization;
- this is done "trivially", in the sense that there is no "specialized" code to run;
- and specifically, it's done by zero-filling storage.

Another possible wording would be "trivially zero-initializable"; for trivially default constructible classes, value initialization always boils down to zero initialization. This could clash with possible future extensions of this trait (in case it is extended to types where value initialization does not perform zero initialization). In general, given that "zero initialization" does not imply "zero filling" (and vice-versa), we would prefer to highlight the latter name and avoid any possible confusion on the intended semantics.

§ 3.4. Future work

A possible future extension to this paper would be to also cover implicit-lifetime types, which are not necessarily trivially *default* constructible. For instance, consider a type like `string_view`:

```
class string_view
{
    const char *begin, *end;

public:
    // not trivial
    constexpr string_view() noexcept : begin(nullptr), end(nullptr) {}
};
```

On all common implementations such as a class is value initializable via zero-filling. `string_view` is also implicit-lifetime: it has a trivial copy constructor and a trivial non-deleted destructor. One can therefore use facilities such as `start_lifetime_as` on a zero-filled storage to create `string_view` objects.

The problem here is that such a detection cannot be automatically done by the compiler, as it can't "see" into the body of a non-trivial default constructor. Therefore, we will necessarily need an opt-in mechanism, such as a type trait or an attribute. This will necessarily complicate the language aspects, with implications similar to e.g. [P1144R6]'s `[[trivially_relocatable]]` attribute.

While we are not proposing such an extension at the moment, it is our belief that this paper should not impede it either.

§ 4. Impact on the Standard

This proposal adds a new property for types to the C++ language, and a corresponding type trait for this property to `<type_traits>`. Vendors are expected to implement the trait through internal compiler hooks.

It is expected that the results of the trait are implementation-specific, as it requires an implementation to consider object representations that are mandated by the architecture/ABI.

§ 5. Technical Specifications

All the proposed changes are relative to [N4892].

§ 6. Proposed wording

Add to the list in [version.syn]:

```
#define __cpp_lib_is_trivially_value_initializable_by_zero_filling YYYYMML // also
```

Add at the end of [basic.types.general]:

12 A type is *trivially value-initializabile by zero-filling* if it is:

- 12.1 an integer type; or
- 12.2 an enumeration type; or
- 12.3 any other scalar type for which it is implementation-defined that it is trivially value-initializabile by zero-filling; or
- 12.4 an array of trivially value-initializabile by zero-filling type; or
- 12.5 a (possibly cv-qualified) trivially value-initializabile by zero-filling class type ([class.prop]).

[*Note 6*: The object representation ([basic.types.general]) of a value-initialized object ([dcl.init.general]) of a trivially value-initializabile by zero-filling type `T` consists of *N* `unsigned char` objects all equal to `0`, where *N* equals `sizeof(T)`. Conversely, it is possible to value-initialize an object of type `T` by filling `N` bytes of suitable storage with zeroes, and starting the lifetime of the `T` object in that storage ([basic.life]). — *end note*]

Add at the end of [class.prop]:

10 A class `S` is a *trivially value-initializabile by zero-filling class* if:

- 10.1 it has an eligible trivial default constructor ([class.default.ctor]), and
- 10.2 all the non-static data members and base classes of `S` are of trivially value-initializabile by zero-filling type ([basic.types.general]).

Modify [meta.type.synop] as shown. At the end of the first [meta.unary.prop] block:

```
  template<class T, class U> struct reference_converts_from_temporary;

  template<class T> struct is_trivially_value_initializable_by_zero_filling;
```

And at the end of the second:

```
  template<class T, class U>
    constexpr bool reference_converts_from_temporary_v
      = reference_converts_from_temporary<T, U>::value;

  template<class T>
    constexpr bool is_trivially_value_initializable_by_zero_filling_v
      = is_trivially_value_initializable_by_zero_filling<T>::value;
```

Add a new row at the end of Table 48 in [meta.unary.prop]:

`template<class T> struct is_trivially_value_initializable_by_zero_filling;`	T is a trivially value-initializabile by zero-filling type ([basic.types.general]).	T shall be a complete type, *cv* `void`, or an array of unknown bound.

7. Acknowledgements

Thanks to KDAB for supporting this work.

Thanks to Ed Catmur for the discussions and drafting a proposal to allow `std::bit_cast` of null pointer values during constant evaluation.

Thanks to Thiago Macieira and Arthur O'Dwyer for the discussions.

All remaining errors are ours and ours only.

References

Informative References

[BOOST-CONTAINER]

Boost.Container 1.81.0: Non-standard value initialization using std::memset. URL: https://www.boost.org/doc/libs/1_81_0/doc/html/container/cpp_conformance.html#container.cpp_conformance.non_standard_memset_initialization

[FOLLY]

IsZeroInitializable type trait. URL: https://github.com/facebook/folly/blob/main/folly/Traits.h#L446

[ITANIUM]

Itanium C++ ABI, 2.3.1 Data Member Pointers. URL: https://itanium-cxx-abi.github.io/cxx-abi/abi.html#data-member-pointers

[LWG2827]

Richard Smith. *is_trivially_constructible and non-trivial destructors*. New. URL: https://wg21.link/lwg2827

[N4892]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 18 June 2021. URL: https://wg21.link/n4892

[P1144R6]

Arthur O'Dwyer. *Object relocation in terms of move plus destroy*. 10 June 2022. URL: https://wg21.link/p1144r6